

DHCP Relays and 3-Tier Architectures

Juan David Guarnizo Hernandez, PhD
juan.guarnizo@tec.mx

TC2006B: Device Communications
Tecnológico de Monterrey
FJ2026

Goals

By the end of this comprehensive exercise you will:

1. Understand native DHCP behavior within a single broadcast domain (Local Subnet).
2. Deploy a 3-Tier Enterprise Campus network (Core, Distribution, Access) in Mininet using OVS.
3. Configure a central `dnsmasq` DHCP server utilizing custom configuration files, including DNS parameters.
4. Implement a DHCP Relay Agent (`dhcrelay`) to transport Layer 2 broadcast requests across a Layer 3 routing boundary.

0) Pre-requirements & Software Installation

To simulate central DHCP services and broadcast relaying, we must install specific Linux networking packages on our host environment. Ensure your system is updated and the necessary daemons are installed:

```
1 sudo apt-get update
2 sudo apt-get install -y mininet dnsmasq isc-dhcp-relay isc-dhcp-client tcpdump
  xterm
```

1) Part 1: Local DHCP in a Single Broadcast Domain

Before introducing complex routing and VLAN boundaries, we will observe how DHCP functions natively within a single Layer 2 broadcast domain. We will use Mininet's default CLI to quickly spin up a single switch connected to three hosts.

1. Start the Basic Topology

```
1 sudo mn --topo single,3 --mac
```

This creates a single default switch (`s1`) with three hosts (`h1`, `h2`, `h3`). We will designate `h1` as our DHCP server.

2. Configure the DHCP Server (`h1`) First, we must assign a static IP to the interface that will serve the DHCP requests. In the Mininet prompt, run:

```
1 mininet> h1 ip addr flush dev h1-eth0
2 mininet> h1 ip addr add 10.0.0.10/24 dev h1-eth0
```

Now, start the `dnsmasq` daemon directly from the CLI. We will provide the configuration directly via command-line arguments instead of a file for this simple test. We configure it to run as a standard background daemon, but we instruct it to log its output to a file (`--log-dhcp`):

```
1 mininet> h1 dnsmasq --dhcp-range=10.0.0.50,10.0.0.150,255.255.255.0,12h --pid-
  file=/tmp/basic_dhcp.pid --log-dhcp --log-facility=/tmp/basic_dhcp.log
```

3. Request IP Addresses (h2 and h3) The remaining hosts currently have no IP addresses. We will use `dhclient` to request them.

```
1 mininet> h2 dhclient -v h2-eth0
2 mininet> h3 dhclient -v h3-eth0
```

Notice that the process succeeds immediately. Because `h1`, `h2`, and `h3` are plugged into the same physical switch on the same native VLAN, the Layer 2 broadcast (DHCPDISCOVER) reaches the server directly without needing a router or relay agent.

4. Clean Up Clean up this basic network before moving to the advanced topology:

```
1 mininet> h1 killall dnsmasq
2 mininet> exit
3 sudo mn -c
```

2) Part 2: Building the 3-Tier Campus with DHCP Relay

In an enterprise environment, it is inefficient to place a DHCP server on every single access switch. Instead, DHCP is centralized. We will now build the architecture shown in Figure 1, introducing VLANs and a Layer 3 boundary.

2.1 Creating the DHCP Configuration File

We must prepare the configuration file that our central DHCP server will use. This defines the IP ranges, gateways, and public DNS servers (like Google's 8.8.8.8) for each distinct VLAN.

Using your preferred text editor, create the configuration file at `/tmp/dhcp_campus.conf` and add the following contents:

```
1 except-interface=lo
2
3 dhcp-range=set:vlan10,10.0.10.50,10.0.10.150,255.255.255.0,12h
4 dhcp-option=tag:vlan10,option:router,10.0.10.254
5 dhcp-option=tag:vlan10,option:dns-server,8.8.8.8
6
7 dhcp-range=set:vlan20,10.0.20.50,10.0.20.150,255.255.255.0,12h
8 dhcp-option=tag:vlan20,option:router,10.0.20.254
9 dhcp-option=tag:vlan20,option:dns-server,8.8.8.8
```

2.2 Python Topology Script

Now we will create a Python script that provisions our Access layer, Distribution layer, and Core layer. The Core switch (`core1`) acts as the Layer 3 gateway for the VLANs and hosts the Relay Agent.

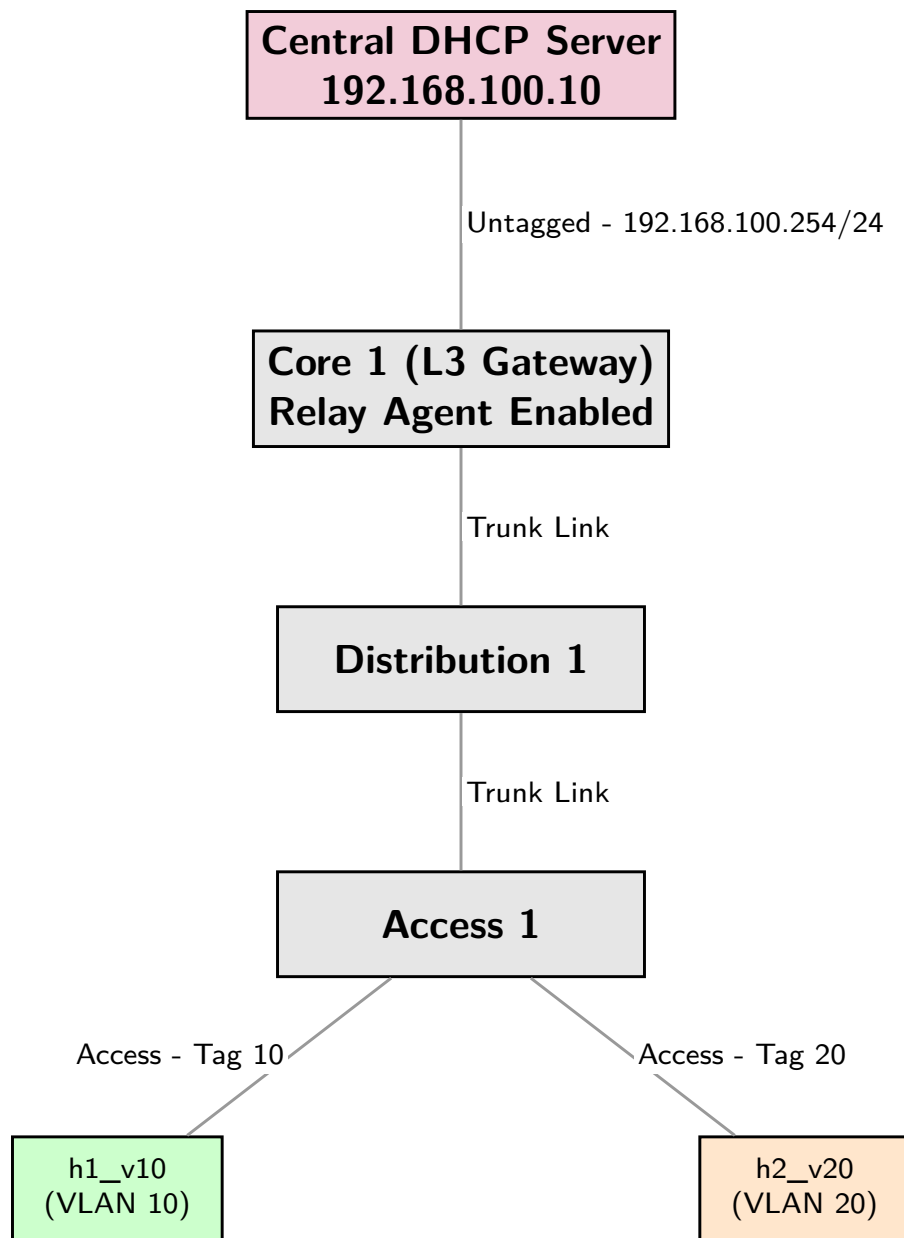


Figure 1: Target Architecture (Part 2): A central DHCP server servicing multiple VLANs via a Core Relay Agent.

Create a file named `campus_dhcp.py` and add the following code:

```

1  #!/usr/bin/env python3
2  from mininet.net import Mininet
3  from mininet.node import OVSSwitch
4  from mininet.link import TCLink
5  from mininet.cli import CLI
6  from mininet.log import setLogLevel
7
8  def build_campus():
9      net = Mininet(controller=None,
10                   switch=OVSSwitch,
11                   link=TCLink,
12                   autoSetMacs=True,
13                   autoStaticArp=False)

```

```

14     core_1 = net.addSwitch('core1')
15     dist_1 = net.addSwitch('dist1')
16     acc_1 = net.addSwitch('acc1')
17
18
19     dhcp_server = net.addHost('dhcpsrv', ip='192.168.100.10/24', defaultRoute='
via 192.168.100.254')
20
21     h1_v10 = net.addHost('h1_v10', ip=None)
22     h2_v20 = net.addHost('h2_v20', ip=None)
23
24     net.addLink(h1_v10, acc_1, port1=0, port2=1)
25     net.addLink(h2_v20, acc_1, port1=0, port2=2)
26     net.addLink(acc_1, dist_1)
27     net.addLink(dist_1, core_1)
28     net.addLink(dhcp_server, core_1, port1=0, port2=2)
29
30     net.start()
31
32     acc_1.cmd('ovs-vsctl set port acc1-eth1 tag=10')
33     acc_1.cmd('ovs-vsctl set port acc1-eth2 tag=20')
34
35     core_1.cmd('ovs-vsctl add-port core1 vlan10 tag=10 -- set interface vlan10
type=internal')
36     core_1.cmd('ip addr add 10.0.10.254/24 dev vlan10')
37     core_1.cmd('ip link set vlan10 up')
38
39     core_1.cmd('ovs-vsctl add-port core1 vlan20 tag=20 -- set interface vlan20
type=internal')
40     core_1.cmd('ip addr add 10.0.20.254/24 dev vlan20')
41     core_1.cmd('ip link set vlan20 up')
42
43     core_1.cmd('ip addr add 192.168.100.254/24 dev core1-eth2')
44     core_1.cmd('sysctl -w net.ipv4.ip_forward=1')
45     core_1.cmd('dhcrelay -4 -i vlan10 -i vlan20 192.168.100.10')
46
47     CLI(net)
48
49     core_1.cmd('killall dhcrelay 2>/dev/null')
50     net.stop()
51
52 if __name__ == '__main__':
53     setLogLevel('info')
54     build_campus()

```

3) Validating the DHCP Relay and DORA Process

Run your script: `sudo python3 campus_dhcp.py`.

We will now act as the client computers powering on for the first time. They currently have no IP addresses configured and must discover a DHCP server dynamically.

3.1 Start the DHCP Server and Monitor Logs

To fully observe the process without locking our terminal, we will run the DHCP server as a standard background daemon, but instruct it to write detailed DHCP events to a log file using `--log-dhcp`. We will then monitor that file.

Start the `dnsmasq` daemon from the Mininet CLI:

```
1 mininet> dhcprsv dnsmasq --conf-file=/tmp/dhcp_campus.conf --pid-file=/tmp/dhcp_campus.pid --dhcp-leasefile=/tmp/dhcp_campus.leases --log-dhcp --log-facility=/tmp/dhcp_campus.log
```

Now, open two xterms for the `dhcprsv` node from the Mininet CLI:

```
1 mininet> xterm dhcprsv dhcprsv
```

In the FIRST dhcprsv terminal, continuously stream the log file using `tail`. This allows you to see the output live:

```
1 # tail -f /tmp/dhcp_campus.log
```

In the SECOND dhcprsv terminal, start a packet capture looking strictly for DHCP/BOOTP traffic (ports 67 and 68):

```
1 # tcpdump -i dhcprsv-eth0 -n -v port 67 or port 68
```

3.2 Request an IP Lease

Open an xterm for your VLAN 10 host from the Mininet CLI:

```
1 mininet> xterm h1_v10
```

Inside the `h1_v10` terminal, force the DHCP client daemon to request a new lease:

```
1 # dhclient -v h1_v10-eth0
```

Observe the output across your screens: 1. **In the first server terminal (tail)**: You will see the live logs indicating an IP was offered and acknowledged. 2. **In the second server terminal (tcpdump)**: You will see the raw DHCPDISCOVER, DHCPOFFER, DHCPREQUEST, and DHCPACK messages flowing.

4) Reflection Questions

1. **Layer 2 Boundaries**: In Section 1, the DHCPDISCOVER broadcast reached the DHCP server without issue. However, in Section 2, the server is on the 192.168.100.x subnet while the client is on the 10.0.10.x subnet. Why is it physically impossible for the original broadcast packet to reach the `dhcprsv` node without the `dhcrelay` daemon running on `core1`?
2. **Packet Analysis**: Look closely at the `tcpdump` output on the DHCP server during the DORA (Discover, Offer, Request, Acknowledge) process. Locate the `giaddr` (Gateway IP Address) field in the payload of the incoming Discover packet. What specific IP address was stamped in the `giaddr` field, and why was this critical for the server to know which IP pool to select?
3. **DNS Configuration**: The DHCP configuration file pushed 8.8.8.8 as the DNS server. Why is it advantageous to assign DNS configurations dynamically via DHCP rather than statically configuring them on every host machine in an enterprise environment?

4. **WAN Scalability:** In the Python script, we deliberately passed the flag `--conf-file=...` to the `dnsmasq` daemon. In a massive simulation linking Campus A and Campus B over a WAN, what would happen if you ran a default DHCP daemon on a server in both campuses without specifying unique configuration and PID files?
5. **High Availability:** If you were to add `core2` to this architecture utilizing HSRP/VRRP (sharing the Virtual IP `10.0.10.254` with `core1`), would you need to run the `dhcrelay` daemon on `core2` as well? Defend your architectural decision based on what happens during a physical hardware failure.